Type 1: Research Project

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

An AI-Driven, Blockchain-Anchored Cyber Deception Framework for Adaptive Honeypot Defense

Team Number: 58

S.No	Register Number	Student Name
1	CB.SC.U4CSE23111	B Guru Sai Prasad
2	CB.SC.U4CSE23127	N Surya Tejaswar
3	CB.SC.U4CSE23129	N Sisir Reddy
4	CB.SC.U4CSE23611	C Pranav Sharma
5	CB.SC.U4CSE23644	R Vishnu Sathwick

Guide: **Dr. Radhika G**

Problem Statement

Traditional honeypots rely on **static, deterministic response tables** — efficient at scale, but fragile against fingerprinting.

- **Fingerprintable by design.** Prior work shows automated probing can distinguish static honeypots (Cowrie, Kippo, Dionaea) from real systems with near-perfect accuracy in a *single packet exchange*.
- **Single-protocol LLM deception.** Existing LLM-backed honeypots (shelLM, HoneyGPT) are concentrated on SSH/terminal sessions only — no shared framework spans multiple heterogeneous protocols.
- **Untrustworthy forensic logs.** A compromised honeypot host can have its own logs silently edited by the attacker, destroying the evidentiary value of captured telemetry.

Objective

To build a deployable, multi-protocol deception framework that stays convincing under sustained probing and produces a tamper-evident audit trail:

- **Cover five distinct attack surfaces** — SSH, an internal AI API, PostgreSQL, an Ethereum JSON-RPC node, and Jenkins CI/CD — behind one shared response engine.
- **Stay fast where it can, smart where it must.** Answer routine traffic (≈80%) at millisecond latency via a static path; reserve LLM generation for genuinely novel input.
- **Make every interaction context-aware.** Replace canned responses with an LLM engine that is session-history-aware and intent-classified.
- **Anchor forensic integrity independently of the host.** Hash and commit every log entry to a separate blockchain so tampering is detectable even if the honeypot itself is compromised.
- **Evaluate honestly.** Measure latency, throughput, response diversity, and realism as separate axes across successively larger and more realistic datasets — and report failures, not just favorable numbers.

Research Gaps We Address

- **LLM deception is single-protocol.** shelLM, HoneyGPT, and related 2023–2025 work apply LLM response generation only to SSH/terminal sessions; no prior system applies one LLM engine uniformly across five heterogeneous protocol types.
- **Tamper-evident logging is generic, not honeypot-specific.** Blockchain hash-anchoring is established in digital forensics (evidence tracking, cloud VM investigation), but we found no honeypot-specific paper combining hash-anchoring with decoy logging.
- **Realism is rarely benchmarked rigorously.** Prior LLM-honeypot papers report a single self-evaluated accuracy figure (e.g. HoneyGPT's self-reported $\approx$96%); few report latency/throughput trade-offs or scale-dependent failure modes.

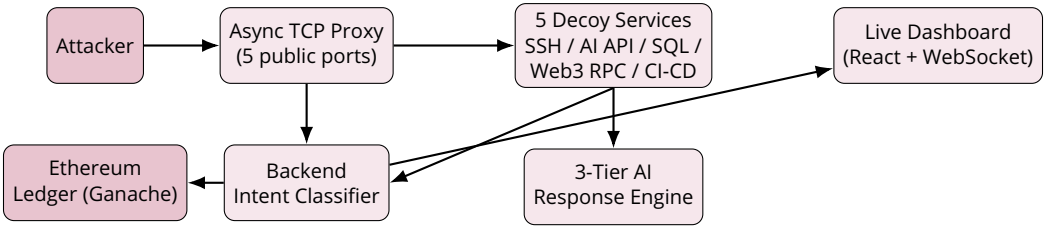
Comparison Against Cited Systems

System	Interaction	AI/LLM Responses	Protocols	Tamper-evident
Cowrie	Medium/High	No (experimental single-tier LLM fallback exists)	SSH, Telnet	No
T-Pot	Mixed (orches-trator)	No – aggregation only	Broad (bundled daemons)	No
Dionaea	Low–Medium	No	SMB, MySQL, SIP, FTP, TFTP	No
shelLM	High (SSH only)	Yes – full LLM shell session	SSH only	Not ad-dressed
HoneyGPT	High (terminal)	Yes – GPT-3.5/4 via Cowrie proxy	SSH/terminal only	Not ad-dressed
This work	**High, all 5**	**Yes – 3-tier, session-aware, uniform across 5 protocols**	**SSH, AI API, SQL, Web3 RPC, CI/CD**	**Yes – SHA-256 + Ethereum anchor**

Table I, paper Section II.F

System Architecture — Overview

Six components; only the proxy and dashboard face the network. All else communicates over an isolated internal Docker network.



Paper Section III

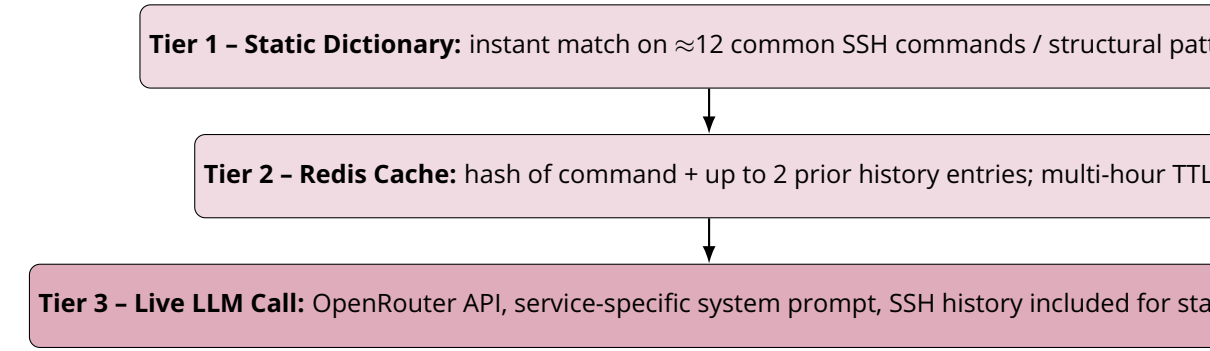
Five Decoy Services

Decoy	Port / Protocol	Role
`shadow-shell`	2222 / SSH	Real `asyncssh` server; accepts any credentials; shell prompt for `dbadmin@corp-prod-db-01`; commands route through AI engine
`oracle-ai`	8080 / REST	FastAPI chat/predict endpoints for a fictitious internal model; prompt-injection bait
`vault-sql`	5433 / Postgres	Real seeded Postgres instance with synthetic records + honeytoken API keys (OpenAI/GitHub/AWS-formatted)
`ledger-trap`	8545 / JSON-RPC	Ethereum node decoy; static replies for common RPC calls, AI engine for uncommon ones
`forge-ci`	9090 / HTTP	Jenkins-style CI/CD endpoints faking a working pipeline; supply-chain bait

Paper Section III.B

Three-Tier AI Response Engine

Expensive LLM calls are reserved for genuinely novel input.



Intent classification: keyword-based, 8 categories (SQLi, privesc, reverse shell, cryptomining, prompt injection, supply-chain, recon, unknown) — a rule-based baseline, reported as a known limitation. *Paper Section III.C–D*

Blockchain-Anchored Logging

- Every log entry (`POST /api/log`) is timestamped, intent-classified, serialized with sorted keys, and **SHA-256 hashed**.
- The hash is anchored via a **zero-value self-transaction** on a local Ethereum chain (Ganache).
- Stored in a bounded Redis list; broadcast over WebSocket to the live dashboard.
- Because the hash lives on an independently operated ledger, any later edit to the stored log is detectable by re-hashing and comparing — **without trusting the honeypot host's own integrity**.

Integrity Test Result

A log entry (hash `356b7496...33a9a1`) was anchored, fetched back, and re-hashed — identical. A tampered copy (`"sudo cat /etc/shadow"` → `"ls -la"`) produced a different hash (`7530fb00...0d9d394`), confirming tamper-evidence.

Paper Section III.E, V.D

Evaluation Methodology

Evaluated against the system's own static fast-path (functionally equivalent to Cowrie) vs. its LLM-backed dynamic path.

- **Four metrics, kept separate:**
  - Latency / throughput (30 static trials, 20 dynamic trials)
  - Response diversity — fraction of distinct outputs per service
  - LLM-judge realism score — independent 70B judge (distinct from 8B generator), 1–5 rubric, 30-payload sample
  - Blockchain integrity — direct tamper test
- **Four successive runs**, each scaling up dataset realism:
  1. 100 hand-written payloads (baseline)
  2. Same dataset, after an optimization pass
  3. 200 real, publicly-documented payloads
  4. 1,328 real payloads (scaled corpora)
- Explicitly **not** a single composite accuracy figure — trade-offs and failures are reported, not hidden.

Paper Section IV

Run 1: Initial Baseline (100 Hand-Written Payloads)

Metric	Static	Dynamic
Mean latency	0.4 ms	4,028 ms (4.03 s)
Median latency	0.4 ms	3,529 ms (3.53 s)
Throughput (req/s)	2,500	0.25
Diversity ratio (mean)	$\approx$0	0.99 (0.95–1.00)

LLM-judge realism: **4.38/5** ($\sigma = 1.28$, $n = 24$ of 30 parsed) *Table II, paper Section V.A*

Run 2: Optimization Pass

Two fixes identified by profiling:

- HTTP connection pooling (was opening a fresh TCP+TLS handshake to OpenRouter *every request*)
- Generation-length cap tuned from 150 $\rightarrow$ 60 tokens (150 tok $\approx$5.3s, 80 tok $\approx$3.4s, 40 tok $\approx$2.0s — generation length dominates latency)
- Static SSH dictionary widened with 12 more recon commands

Metric	Before	After	Change
Dynamic mean latency	4.03 s	1.35 s	$\approx 3\times$
Dynamic throughput (seq.)	0.25 rps	0.74 rps	$\approx 3\times$
Static throughput	2,500 rps	3,333 rps	$+33\%$
LLM-judge realism	4.38/5 (24/30)	4.00/5 (30/30)	-0.38

Table III, paper Section V.B

Latency Comparison — Before vs. After

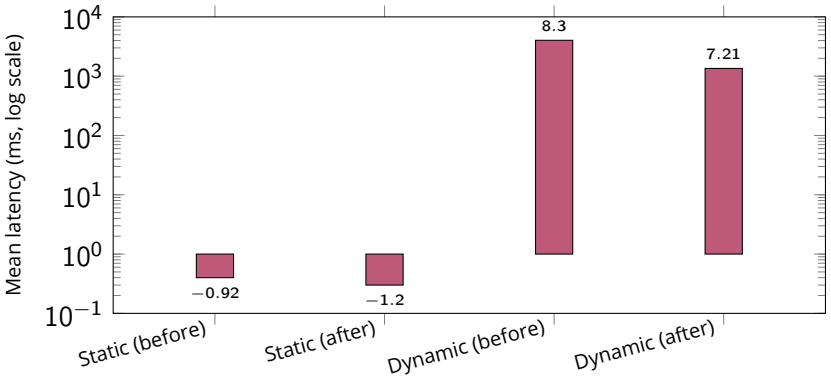


Fig. 1, paper Section V.B

Decoy-Layer Connection Pooling (All 5 Decoys)

Decoy	Before (mean)	After (mean)	Δ
oracle-ai	6.15 ms	4.28 ms	−30.4%
vault-sql	0.69 ms	0.41 ms	−41.4%
ledger-trap	5.96 ms	4.20 ms	−29.5%
forge-ci	6.95 ms	6.71 ms	−3.4%
shadow-shell	7.91 ms	4.64 ms	−41.3%

Real and consistent improvement on 4 of 5 decoys; `forge-ci` barely moves since it always routes through the AI engine regardless of decoy-layer pooling. *Table IV, paper Section V.C*

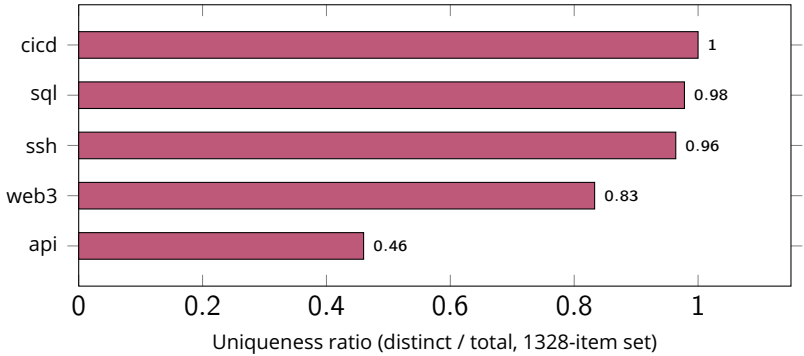
Runs 3–4: Real-World Payload Datasets

The hand-written dataset was replaced with **real, publicly-documented** attack payloads (payload-box, PayloadsAllTheThings, SecLists), scaled 100 → 200 → 1,328 payloads.

Metric	200-item	1,328-item
Dynamic latency (mean)	2.06 s	1.95 s
Concurrent throughput (10 sessions)	1.62 req/s	3.05 req/s
Response diversity – ssh	0.925	0.964
Response diversity – sql	1.00	0.978
Response diversity – web3	0.80	0.833
Response diversity – cicd	0.90	1.00
Response diversity – **api**	**0.70**	**0.46**
LLM-judge realism (mean)	4.03/5	3.37/5

Table VI, paper Section V.E–F

Response Diversity by Decoy Service



api is the clear outlier — the model frequently collapses to a generic *"invalid request"* reply on real XXE / Log4Shell strings instead of emulating a plausible vulnerable-parser response. *Fig. 5, paper Section V.C, V.F*

The Benchmark-Harness Bug

A genuine measurement bug was found and fixed during the real-dataset re-run.

- The concurrent-throughput benchmark reused payloads **already sent once earlier** in the same run.
- Because the Redis cache key derives from command + history, these were guaranteed cache **hits** — so the “800 req/s” figure measured Redis latency, not LLM latency.
- **Fix:** append a fresh UUID nonce to every payload immediately before firing the concurrent batch.

Metric	First-reported (bug)	Corrected
Concurrent throughput (10 sessions)	800.3 req/s	3.05 req/s
Mean per-request latency	8.2 ms	2.62 s

Reported explicitly rather than silently substituted — a 3-order-of-magnitude bug is itself a methodological finding. *Table V, paper Section V.E*

Realism Score Distribution (Post-Optimization, n=30)

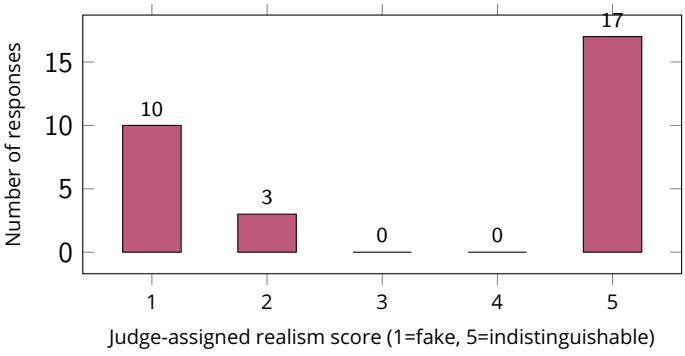


Fig. 3, paper Section V.B

Key Finding: The API Decoy's Realism Failure

At the 1,328-item scale, **9 of 11** sampled `api` responses scored 1–2/5 — versus near-uniform 5/5 for ssh, sql, and web3 in the same sample.

Most Consequential Result

One sampled response was found to contain an **explicit AI safety disclaimer** (*"Contains an explicit disclaimer about not generating harmful responses"*) — a hard deception failure, not a minor realism deduction. It signals *"this is not a real system"* to exactly the attentive attacker the framework is designed to hold.

- This failure was **invisible** in the original 100-payload and even the 200-payload runs — it only became unambiguous at the largest (1,328-item) scale.
- Root cause: the hosted model's own safety training bleeds through the deception persona on XML/JNDI-shaped adversarial input.
- **Actionable fix (not yet applied):** XXE/Log4Shell/SSRF-aware few-shot examples + explicit anti-disclaimer instruction in `oracle-ai`'s system prompt.

Paper Section V.F, VI

Discussion

- **Not a drop-in replacement** for high-volume, low-interaction scanning honeypots — static-path systems remain appropriate there.
- **Targeted use case:** high-interaction deception against a human operator or a sophisticated, low-volume tool, where multi-second latency is plausibly attributable to network/server load.
- **Concurrency is real but modest:** 10 concurrent sessions complete in 3.28 s wall-clock (not ≈19.5 s), but this is far short of the originally (buggy) reported capacity.
- **The AI-persona failure is structural, not incidental:** any decoy built on a hosted, safety-tuned model inherits an alignment layer that is directly adversarial to the deception goal on concerning inputs.
- Blockchain anchoring assumes the ledger is outside attacker reach — reasonable for an internal Ganache deployment, not evaluated against an adversary who also compromises the chain client.

Paper Section VI

Limitations & Future Work

- LLM-judge realism score is an **automated proxy** — needs human-subject validation (protocol specified in paper Appendix A, not yet executed).
- Real-payload datasets (up to 1,328 items) are still **not live-traffic captures** — no multi-step sessions, timing, or evasion behavior.
- `web3` and `cicd` datasets remain small (18, 10 payloads) — no comparably large real corpus exists for those surfaces; a genuine coverage gap.
- Intent classification is **keyword-based** — will misclassify obfuscated/novel patterns; a learned classifier is a natural extension.
- No longitudinal live-internet deployment yet; concurrent-throughput tested only at a single burst level (10 sessions).
- **Immediate next step:** rewrite `oracle-ai`'s system prompt with XXE/JNDI-aware few-shot examples and an explicit anti-disclaimer instruction, then re-measure.

Paper Section VII

References I


A. Vetterl and R. Clayton, “Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale,” *Proc. 12th USENIX WOOT*, 2018.


“Gotta Catch ’em All: A Multistage Framework for Honeypot Fingerprinting,” *ACM Digital Threats: Research and Practice*, 2023, arXiv:2109.10652.


M. Sladić, V. Valeros, C. Catania, and S. Garcia, “LLM in the Shell: Generative Honeypots,” *Proc. IEEE EuroS&PW*, 2024, arXiv:2309.00155.


Z. Wang et al., “HoneyGPT: Breaking the Trilemma in Terminal Honeypots with Large Language Model,” arXiv:2406.01882, 2024.


F. McKee and D. Noever, “Chatbots in a Honeypot World,” arXiv:2301.03771, 2023.


“LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems,” arXiv:2409.08234, 2024.


“Blockchain-Based Evidence Tracking System for Forensic Integrity and Secure Chain of Custody,” IEEE, 2025.


“Blockchain-based tamper-proof and transparent investigation model for cloud VMs,” *The Journal of Supercomputing*, Springer, 2022.


“A novel intelligent cyber threat detection system with blockchain-enforced immutable forensic auditability,” *PeerJ Computer Science*, 2025.

References II


MITRE, “MITRE Engage: A Framework and Community for Cyber Deception,” 2022.


Cowrie SSH/Telnet Honeypot, GitHub repository.


“SoK: Honeypots and Large Language Models — Systematization of a Growing Intersection,” arXiv:2510.25939, 2025.

Thank You